

Autonomous Emergency Braking with Sensor Fusion

ON THIS PAGE

Introduction

Explore Test Bench Model

Model AEB Controller

Explore Metrics Assessment

Simulate AEB Model

Generate C++ Code

Explore Additional Scenarios

References

See Also

This example shows how to implement autonomous emergency braking (AEB) using a sensor fusion algorithm.

Introduction

Autonomous emergency braking is an advanced active safety system that helps drivers avoid or mitigate collisions with other vehicles.

The European New Car Assessment Programme (Euro NCAP®) has included the AEB city and interurban systems in its safety rating since 2014. The Euro NCAP continues to promote AEB systems for protecting vulnerable road users, such as pedestrians and cyclists.

Today, AEB systems mostly use radar and vision sensors to identify potential collision partners ahead of the ego vehicle. These systems often require multiple sensors to obtain accurate, reliable, and robust detections while minimizing false positives. To combine the data from various sensors, multiple sensor AEB systems use sensor fusion technology. This example shows how to implement AEB using a sensor fusion algorithm. In this example, you:

1. **Explore the test bench model** — The model contains the sensors and environment, sensor fusion and tracking, decision logic, controls, and vehicle dynamics.
2. **Model the AEB Controller** — Use Simulink® and Stateflow® to integrate a braking controller for braking control and a nonlinear model predictive controller (NLMPCC) for acceleration and steering controls.
3. **Simulate the test bench model** — You can configure the test bench model for different scenarios based on Euro NCAP test protocols.
4. **Generate C++ code** — Generate C++ code and test the software-in-the-loop (SIL) simulation for the sensor fusion, decision logic, and control algorithms.
5. **Explore additional scenarios** — These scenarios test the system under additional conditions.

Explore Test Bench Model

In this example, you use a system-level simulation test bench model to explore the behavior of the controller for an AEB system.

To explore the test bench model, load the autonomous emergency braking project.

```
openProject("AutonomousEmergencyBraking");
```



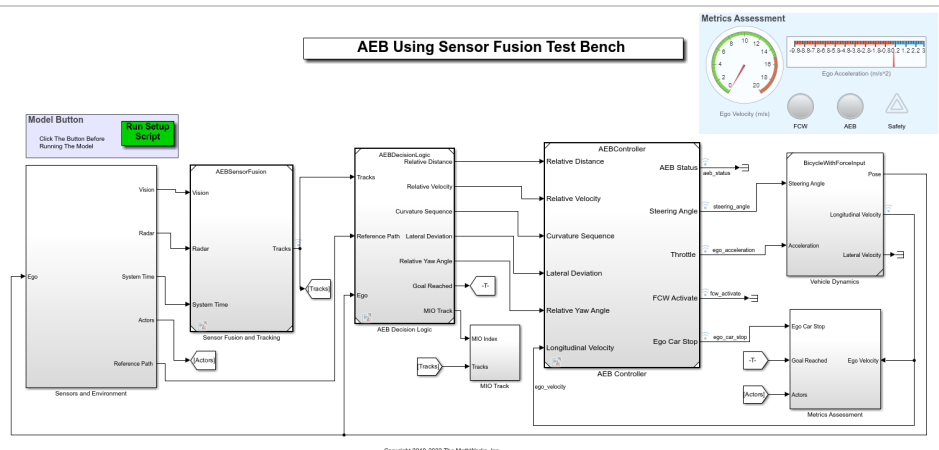
To reduce Command Window output, turn off model predictive controller (MPC) update messages.

```
mpcverbosity("off");
```



Open the system-level simulation test bench model.

```
open_system("AEBTestBench");
```



Opening this model runs the `helperSLAEBSetup` helper function, which initializes the scenario using the `drivingScenario` object in the base workspace. It runs the default test scenario, `scenario_25_AEB_PedestrianTurning_Nearside_10kph`, that contains an ego vehicle and a pedestrian. This setup function also configures the controller design parameters, vehicle model parameters, and Simulink bus signals required for defining the inputs and outputs for the `AEBTestBench`

This example uses:

[Automated Driving Toolbox](#)

[Model Predictive Control Toolbox](#)

[Optimization Toolbox](#)

[Simulink](#)

Copy Command

model.

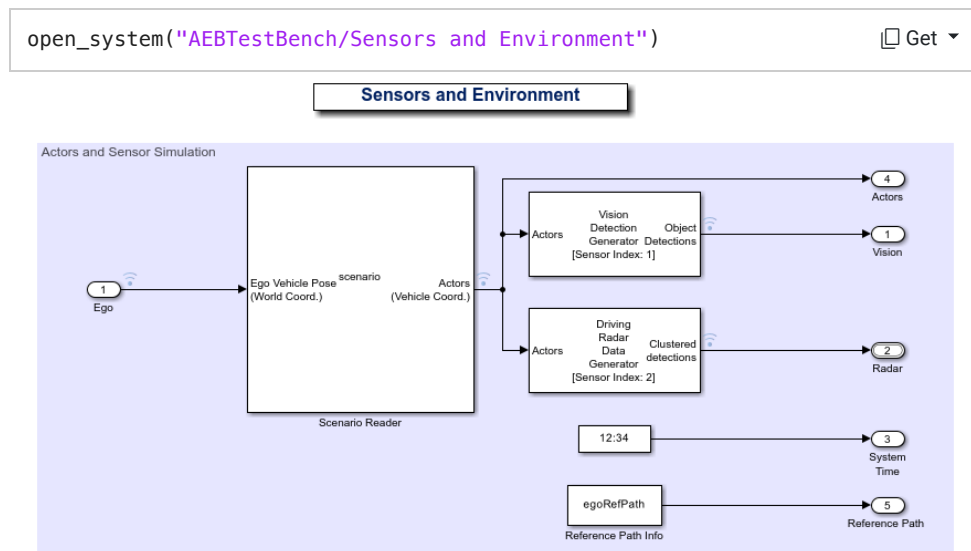
The test bench model contains these modules:

- **Sensors and Environment** – Subsystem that specifies the road, actors, camera, and radar sensor used for simulation.
- **Sensor Fusion and Tracking** – Algorithm model that fuses vehicle detections from the camera to those from the radar sensor.
- **AEB Decision Logic** – Algorithm model that specifies the lateral and longitudinal decision logic that provides most important object (MIO) related information and ego vehicle reference path information to the controller.
- **AEB Controller** – Algorithm model that specifies the steering angle and acceleration controls.
- **Vehicle Dynamics** – Subsystem that specifies the dynamic model of the ego vehicle.
- **Metrics Assessment** – Subsystem that assesses system-level behavior.

The **Vehicle Dynamics** subsystem models the ego vehicle using a Bicycle Model, and updates its state using commands received from the **AEB Controller** model. For more details on the **Vehicle Dynamics** subsystem, see the [Highway Lane Following](#) example.

To plot synthetic sensor detections, tracked objects, and ground truth data, use the [Bird's-Eye Scope](#). The **Bird's-Eye Scope** is a model-level visualization tool that you can open from the Simulink model toolbar. On the **Simulation** tab, under **Review Results**, click **Bird's-Eye Scope**. After opening the scope, click **Update Signals** to set up the signals. The dashboard panel displays these ego vehicle parameters: velocity, acceleration, AEB status, forward collision warning (FCW) status, and safety status.

The **Sensors and Environment** subsystem configures the road network, defines target actor trajectories, and synthesizes sensors. Open the **Sensors and Environment** subsystem.



The subsystem specifies the scenario and sensors of the ego vehicle using these blocks:

- The [Scenario Reader](#) block reads the `drivingScenario` object from the base workspace, and then reads the actor data from that object. The block uses the ego vehicle information to perform a closed-loop simulation, and then outputs the ground truth information of the scenario actors and their trajectories in ego vehicle coordinates.
- The [Driving Radar Data Generator](#) block generates radar sensor data from a driving scenario.
- The [Vision Detection Generator](#) block generates detections and measurements from a camera mounted on the ego vehicle.
- The [Reference Path Info](#) block provides a predefined reference trajectory for ego vehicle navigation. The reference path in the block is created by using the `helperEgoRefPathCreator` helper function.

The **Sensor Fusion and Tracking** algorithm model processes vision and radar detections and generates the position and velocity of the tracks relative to the ego vehicle. Open the **AEBSensorFusion** algorithm model.

```
open_system("AEBSensorFusion");
```